

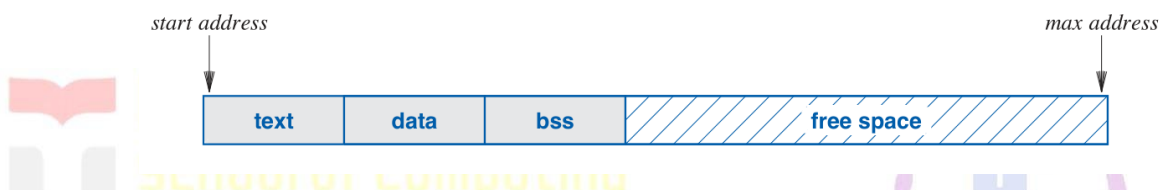
Modul 11 Memori Xinu

Tujuan Praktikum

1. Praktikan mampu memahami struktur memori Xinu.

11.1 Memori

1. Pada Xinu, memory image dibagi menjadi 4 bagian:
 - a. Text segment: berisi kode yang telah dcompile dalam bahasa mesin (instruksi mesin) untuk semua fungsi. Hasil kompilasi fungsi printf(), perintah-perintah shell berada pada segmen ini. Text segmen berada pada bagian alamat memory paling bawah.
 - b. Data segment: berisi semua variable global yang telah diinisiasi nilainya, berada setelah text segment.
 - c. Bss segment (block started by symbol segment): berisi global variable yang tidak diinisiasi, berada setelah data segment.
 - d. Free space: alamat setelah bss segment



Gambar 11-1 Bagian Memory Image Pada Xinu

2. Image di atas adalah ilustrasi image Xinu pada saat belum berjalan (static). Apa yang terjadi jika Xinu berjalan/booting? Pada saat booting maka akan dibuat proses-proses baru. Proses-proses baru tersebut akan menempati free space yang ada. Setiap kali ada proses baru akan dialokasikan free space yang ada dan ketika proses terminasi maka alokasi proses tersebut pada free space akan dihapus. Adanya alokasi dan dealokasi pada free space sangat dinamis tergantung dari kekomplekan kode.
3. Bagian **free space** pada memory dapat digunakan untuk alokasi dinamis memory. Terdapat 2 jenis dinamis memori:
 - a. Stack: pada saat suatu fungsi dipanggil maka akan dibuatkan stack untuk fungsi tersebut. Pada saat fungsi telah selesai maka stack akan dihapus.
 - b. Heap: digunakan ketika fungsi meminta/membuat/membutuhkan alokasi memori baru (pada c misalkan menggunakan perintah malloc()). Contoh: pada saat membaca file, konten file disimpan dahulu pada memori. Untuk menyimpan konten membutuhkan memori sehingga perlu dialokasikan memori heap menggunakan malloc().

Berikut adalah perbedaan lain antara stack dan heap. Stack dialokasikan dari alamat maksimum memori, sedangkan Heap dialokasikan dari alamat terendah dari free space. Stack terstruktur yaitu LIFO sedangkan heap bebas dialokasikan dan dealokasi kapanpun.

4. Ilustrasi stack dan heap



Gambar 11-2 Ilustrasi Memori

Gambar di atas merupakan ilustrasi memori setelah 3 buah proses dibuat dan salah satu proses membuat alokasi memori. Pada saat proses 1 dibuat maka proses 1 berada pada stack 1. Ketika proses 2 dibuat maka proses 2 berada pada stack 2. Begitu juga untuk proses 3 sehingga proses 3 berada pada stack 3. Perhatikan bahwa stack dialokasikan dari alamat memori paling tinggi menuju alamat yang rendah.

Misalkan proses 3 membutuhkan memori heap. Xinu akan mengalokasikan heap mulai dari alamat free memory yang paling rendah. Begitu seterusnya hingga free space tidak mampu menampung lagi stack atau heap.

5. Implementasi manajemen memori pada Xinu

Pada Xinu terdapat empat fungsi utama untuk manajemen memori:

- getstk() - alokasi stack pada saat proses dibuat (create)
- freestk() - release stack pada saat proses terminasi
- getmem() - alokasi heap sesuai permintaan
- freemem() - release heap sesuai permintaan

- Alokasi dan dealokasi stack hanya terjadi pada saat proses dibuat dan diterminasi. Hal ini menjamin bahwa dealokasi stack pasti terjadi pada saat proses terminasi. Akan tetapi, ada permasalahan pada dealokasi heap. Pada Heap, saat proses yang meminta alokasi heap terminasi maka heap tidak secara otomatis dihapus. Proses tersebut harus secara manual dan eksplisit menghapus heap yang dibuat proses tersebut. Masalah lain pada heap adalah permintaan akan heap bisa melebihi free space yang ada. Heap juga akan mengalami fragmentasi.

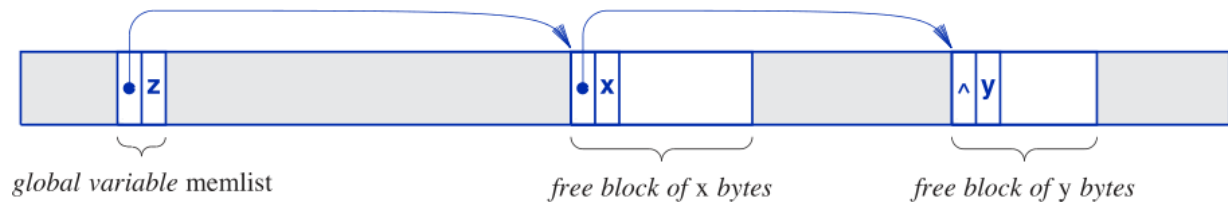
- Untuk mengatasi permasalahan tersebut maka Xinu harus menyimpan informasi mengenai semua free block memory. Xinu menggunakan list yang mencatat awal memori dan ukuran memori yang kosong.

Block	Address	Length
1	0x84F800	4096
2	0x850F70	8192
3	0x8A03F0	8192
4	0x8C01D0	4096

Gambar 11-3 Contoh List Memori Pada Xinu

Jika ada proses yang membutuhkan memori maka Xinu akan mencari pada list tersebut, menentukan awal alamat yang bisa digunakan, mengalokasikan block sesuai permintaan dan mengupdate list tersebut.

8. Xinu menggunakan linked list untuk manajemen memory. Terdapat global variabel Bernama *memlist* yang berisi pointer ke free block kosong pertama. Kemudian *memlist* akan menunjuk ke alamat free block kosong selanjutnya. Pada free block ini akan menunjukkan ukuran free block dan free block selanjutnya.



Gambar 11-4 Ilustrasi Penggunaan Linked List Untuk Manajemen Memory



Fakultas Informatika
School of Computing
Telkom University



informatics lab